



SMART CONTRACT SECURITY ASSESSMENT

Buoy

SECURITY REVIEW

4 SEPTEMBER 2026

rt contracts, our shielding · Your smart contracts, our shielding · Your smart contracts, our shielding · Your smart contracts

Contents

1	About Shieldify	3
2	Disclaimer	3
3	About Buoy	3
4	Risk Classification	4
4.1	Impact	4
4.2	Likelihood	4
5	Security Review Summary	5
5.1	Protocol Summary	5
5.2	Scope	5
6	Findings Summary	5
7	Findings	7

1 About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2 Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3 About Buoy

Buoy is a non-custodial vault platform built on Hyperliquid that lets depositors allocate USDC to experienced perpetuals traders without surrendering custody of their funds. A trader, referred to as a Leader, creates a vault and sets a performance fee. Depositors supply USDC and receive the vault's ERC-20 share token in return, while the Leader trades the pooled capital on Hyperliquid's markets. The vault's net asset value, and therefore the share price, tracks the Leader's trading performance.

The protocol spans two layers. Its contracts live on HyperEVM, while trading occurs on HyperCore, Hyperliquid's native trading engine, with each vault owning a dedicated HyperCore account. USDC is bridged between the layers as deposits settle and withdrawals are paid, and both directions are restricted at the contract level to the vault's own accounts, so there is no code path that routes vault funds to a third-party address.

The main functionalities of the protocol are:

- **Router** – the single gateway for every user action: deposit requests, withdrawal requests, cancellations, and vault creation. It validates requests, escrows funds and shares into the target vault within the same transaction, assigns each request a global ID, collects the fixed request fee, and is the only address permitted to trigger settlement on a vault.
- **VaultFactory** – deploys new vaults and maintains the canonical on-chain registry of legitimate Buoy vaults, which the Router consults before touching any vault. It also curates the builder whitelist used for builder-fee approvals.
- **BuoyVault** – one contract per vault, deployed as a minimal-proxy clone and immutable once live. Each vault is simultaneously the ERC-20 share token, the custodian of the vault's EVM-side USDC, and the bookkeeper for pending requests, withdrawal obligations, the high-water mark, and the performance fee.
- **Asynchronous request lifecycle** – deposits and withdrawals are not settled inline. Users enqueue requests that an off-chain fulfiller later settles against a NAV snapshot submitted together with the vault's current interaction nonce, minting shares, locking and paying withdrawal obligations, and bridging funds between HyperEVM and HyperCore.
- **Fee model** – Leaders earn a performance fee only on new profit highs, enforced through a per-vault high-water mark. There is no management fee.

4 Risk Classification

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	 CRITICAL	 HIGH	 MEDIUM
Likelihood: Medium	 HIGH	 MEDIUM	 LOW
Likelihood: Low	 MEDIUM	 LOW	 LOW

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable, and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5 Security Review Summary

The security review lasted 9 days with a total of 144 hours dedicated to the audit by the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying seven Medium and five Low severity issues. They're mainly related to zero-supply and wind-down accounting edge cases, asymmetries in the asynchronous request lifecycle, grieving vectors that can stall settlement or block router rotation, and assumptions in the HyperEVM-HyperCore integration.

The Buoy team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

PROJECT NAME	BUOY
Repository	buoy-contracts
Type of Project	Non-Custodial Perpetuals Vault Platform (Hyperliquid / HyperEVM)
Security Review Timeline	9 days
Review Commit Hash	80f288527c10285e78f9fabbeedd8890c0e58fa7
Fixes Review Commit Hash	0298afbdb5c424fc82bd113af8c93ffc2bcefa5c

5.2 Scope

The following smart contracts were in the scope of the security review:

FILE	NSLOC
contracts/vault/VaultFactory.sol	154
contracts/vault/BuoyVault.sol	585
contracts/vault/Router.sol	454
Total	1193

6 Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: 7
- **Low** issues: 5

ID	TITLE	SEVERITY	STATUS
[M-01]	Zero-Supply Bootstrap Lets First Depositor Capture Pre-Existing Vault Assets	 MEDIUM	ACKNOWLEDGED
[M-02]	Locked Withdrawal Cancellation Re-Mints Stale Shares After NAV Changes	 MEDIUM	FIXED
[M-03]	Direct Share Transfers to the Vault Can Block Router Rotation	 MEDIUM	FIXED
[M-04]	Public Requests Can Indefinitely Invalidate Fulfiller NAV Snapshots	 MEDIUM	FIXED
[M-05]	Unfulfillable User Requests Can Block Router Rotation	 MEDIUM	FIXED
[M-06]	Stale High-Water Mark Suppresses Fees After Vault Recapitalization	 MEDIUM	FIXED
[M-07]	First-Depositor Share-Price Inflation via Missing Dead Shares at Bootstrap	 MEDIUM	ACKNOWLEDGED
[L-01]	Asynchronous Bridge Accounting Allows Deposits to Mint Excess Shares	 LOW	FIXED
[L-02]	HIP-3 Transfer Path Hardcodes USDC for Non-USDC Collateral DEXes	 LOW	FIXED
[L-03]	Zero NAV Cannot Be Published, Leaving Stale Positive Accounting After Total Loss	 LOW	FIXED
[L-04]	Named HyperCore Agents Can Remain Authorized After Rotation	 LOW	ACKNOWLEDGED
[L-05]	Enabling the Performance Fee Charges it Retroactively on All Prior Gains	 LOW	FIXED

7 Findings

M-01

MEDIUM

ACKNOWLEDGED

Zero-Supply Bootstrap Lets First Depositor Capture Pre-Existing Vault Assets

Description

When `totalSupply() == 0`, `BuoyVault.fulfillDepositRequest()` enters a bootstrap branch that mints shares 1:1 with the deposit amount. This branch ignores any positive pre-deposit `currentNav`, only requiring `totalOwedUsdc == 0` and a fresh nonce.

If the vault has positive NAV while share supply is zero, the next fulfilled depositor becomes the sole shareholder of all vault assets. This can happen before the first LP deposit, after a full wind-down with residual assets, or when delayed bridge credits, donations, trading proceeds, or rounding dust remain after all shares are burned.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function fulfillDepositRequest( uint256 localId, uint256 currentNav, uint256 expectedNonce ) external override
onlyRouter nonReentrant whenNotPaused returns (uint256 sharesToUser) {
    // code
    if (supply == 0) {
        require(totalOwedUsdc == 0, "VAULT: wound down");
        require(expectedNonce == interactionNonce, "VAULT: stale snapshot");
        sharesToUser = amount;
        latestNav = currentNav + amount;
        latestNavTimestamp = uint64(block.timestamp);
        _bumpInteractionNonce();
    } else {
        require(currentNav > 0, "VAULT: nav=0");
        uint256 netNavBefore = _netNav(currentNav);
        require(netNavBefore > 0, "VAULT: net nav=0");
        _crystallizeFee(currentNav, supply);
        uint256 postSupply = totalSupply();
        uint256 netNav = _netNav(currentNav);
        sharesToUser = (amount * postSupply) / netNav;
        _cacheNav(currentNav + amount, expectedNonce);
    }
    // code
}
```

File: [contracts/vault/BuoyVault.sol](#)

```
function lockOwed( uint256 localId, uint256 currentNav, uint256 expectedNonce ) external override onlyRouter
nonReentrant whenNotPaused returns (uint256 owedUsdc) {
    // code
    owedUsdc = (uint256(r.shares) * netNav) / postSupply;
    totalOwedUsdc += owedUsdc;
    _burn(address(this), r.shares);
    // code
}
```

File: [contracts/vault/VaultFactory.sol](#)

File: [contracts/vault/BuoyVault.sol](#)

```
// Factory transfers an operational seed and initialize bridges it, but no
// shares are minted for that value. Supply remains zero until the first LP.
IERC20Upgradeable(usdc).safeTransfer(vault, VAULT_CREATION_SEED_USDC);
_bridgeAmount(p.seedAmount);
```

Impact

A public depositor can acquire ownership of all positive NAV in a zero-supply vault by depositing the minimum amount and waiting for normal fulfillment. They can then withdraw all shares and receive the pre-existing capital once the fulfiller bridges enough USDC back to the vault.

Proof of Concept

1. Vault state is `totalSupply() == 0`, `totalOwedUsdc == 0`, and truthful gross NAV is `1,000,000 USDC`.
2. Attacker requests a `1 USDC` deposit through the Router.
3. Fulfiller calls `fulfillDeposit` with `currentNav = 1,000,000 USDC`.
4. Because supply is zero, the vault mints `1` share to the attacker and sets `latestNav = 1,000,001 USDC`.
5. Attacker owns 100% of the share supply.
6. Attacker requests withdrawal of all shares.
7. `lockOwed` calculates owed value as `shares * netNav / supply`, resulting in approximately `1,000,001 USDC`, then burns the shares.
8. Once the fulfiller bridges liquidity back to EVM, `fulfillWithdrawalRequest` pays the attacker the locked amount.

Recommendation

Prevent 1:1 bootstrap when verified pre-deposit net NAV is nonzero. For example, in the `supply == 0` branch require `_netNav(currentNav) == 0` before minting `sharesToUser = amount`.

Team Response

Acknowledged.

M-02

MEDIUM

FIXED

Locked Withdrawal Cancellation Re-Mints Stale Shares After NAV Changes

Description

When a withdrawal is locked, `lockOwed` fixes the user's USDC entitlement, adds it to `totalOwedUsdc`, and burns the escrowed shares. After the request timeout, the same user can cancel the locked withdrawal. The cancellation removes the fixed USDC obligation but mints back exactly the original number of shares, without repricing those shares against the current vault NAV.

This gives the withdrawer an asymmetric option. If NAV falls, they can keep pursuing the locked USDC payout, subject to any allowed `reduceOwedUsdc()` adjustment. If NAV rises before timeout expiry or before fulfillment, they can cancel and re-enter with the historical share amount, diluting continuing LPs.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function lockOwed(
    uint256 localId,
    uint256 currentNav,
    uint256 expectedNonce
) external override onlyRouter nonReentrant whenNotPaused returns (uint256 owedUsdc) {
    WithdrawalRequest storage r = _withdrawals[localId];
    require(r.status == WithdrawStatus.Pending, "VAULT: not pending");

    uint256 supply = totalSupply();
    uint256 netNavBefore = _netNav(currentNav);
    require(netNavBefore > 0, "VAULT: net nav=0");

    _crystallizeFee(currentNav, supply);
    uint256 postSupply = totalSupply();
    uint256 netNav = _netNav(currentNav);
    owedUsdc = (uint256(r.shares) * netNav) / postSupply;
    _cacheNav(currentNav, expectedNonce);

    r.owedUsdc = _toUint128(owedUsdc);
    r.status = WithdrawStatus.Locked;
    totalOwedUsdc += owedUsdc;
```

```

    _burn(address(this), r.shares);
}

function cancelWithdrawalRequest(
    uint256 localId,
    address caller,
    uint256 timeout
) external override onlyRouter nonReentrant {
    WithdrawalRequest storage r = _withdrawals[localId];
    WithdrawStatus s = r.status;
    require(
        s == WithdrawStatus.Pending || s == WithdrawStatus.Locked,
        "VAULT: cannot cancel"
    );

    uint256 shares = r.shares;
    if (s == WithdrawStatus.Pending) {
        _transfer(address(this), r.user, shares);
    } else {
        uint256 owed = r.owedUsdc;
        totalOwedUsdc -= owed;
        _mint(r.user, shares);
    }

    r.status = WithdrawStatus.Cancelled;
}

```

Impact

A locked withdrawer can capture upside generated while their position was represented by a fixed USDC obligation. The gain is transferred from continuing LPs through dilution when the old share amount is re-minted after NAV has increased.

Proof of Concept

Initial state:

- Vault gross NAV: 100 USDC
- Total supply: 100 shares
- Attacker owns 50 shares
- Other LPs own 50 shares

Steps:

1. Attacker requests withdrawal of 50 shares.
2. Fulfiller locks the request at NAV 100.
 - $\text{owedUsdc} = 50$
 - $\text{totalOwedUsdc} = 50$
 - Attacker's 50 escrowed shares are burned
 - Total supply becomes 50
3. Vault NAV rises to 150 before the locked request is fulfilled.
 - Net NAV for remaining LPs is $150 - 50 = 100$
 - Other LPs' 50 shares are worth 100
4. Timeout expires and attacker cancels the locked request.
 - totalOwedUsdc decreases from 50 to 0
 - Attacker receives the original 50 shares
 - Total supply returns to 100
5. Final state:
 - Vault gross NAV: 150
 - Attacker owns $50 / 100 = 50\%$, worth 75
 - Other LPs own $50 / 100 = 50\%$, worth 75

The attacker gains 25 USDC of upside from a period where they had a fixed 50 USDC withdrawal claim. That value is taken from continuing LPs.

Recommendation

Do not allow user cancellation after a withdrawal reaches Locked. Cancellation should only be available for Pending withdrawals.

If cancellation after locking must remain available, it must use a fresh nonce-protected NAV and convert the cancelled owedUsdc back into shares at the current net price instead of restoring r.shares .

Team Response

Fixed.

Direct Share Transfers to the Vault Can Block Router Rotation

Description

`BuoyVault` uses `balanceOf(address[this]) == 0` as proof that there are no Pending withdrawal shares escrowed in the vault. This assumption is unsafe because the vault share token inherits ordinary ERC20 transfer behavior and does not block direct transfers to the vault address.

In the intended flow, `acceptWithdrawalRequest` transfers shares from the user to the vault and records a matching `_withdrawals[localId]` entry. Later, `lockOwed` burns those escrowed shares, or `cancelWithdrawalRequest` returns them to the user.

However, any shareholder can bypass this accounting by calling `transfer(address[vault], amount)` directly. This increases the vault's share balance without creating a withdrawal record. The orphaned share balance cannot be cancelled, locked, burned, or released through the normal router paths, causing `setRouter` to revert indefinitely with `VAULT: withdrawals pending`.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function acceptWithdrawalRequest(
    address user,
    uint256 shares,
    uint256 minOut
) external override onlyRouter nonReentrant whenNotPaused returns (uint256 localId) {
    require(user != address[0], "VAULT: user=0");
    require(shares > 0, "VAULT: shares=0");
    require(balanceOf(user) >= shares, "VAULT: insufficient shares");

    _transfer(user, address[this], shares);
    _bumpInteractionNonce();
    unchecked {
        localId = ++nextLocalRequestId;
    }
    _withdrawals[localId] = WithdrawalRequest({
        user: user,
        shares: _toUint128(shares),
        owedUsdc: 0,
        minOut: _toUint128(minOut),
        createdAt: uint64(block.timestamp),
        status: WithdrawStatus.Pending
    });
}

function cancelWithdrawalRequest(
```

```

uint256 localId,
address caller,
uint256 timeout
) external override onlyRouter nonReentrant {
    WithdrawalRequest storage r = _withdrawals[localId];
    WithdrawStatus s = r.status;
    require(
        s == WithdrawStatus.Pending || s == WithdrawStatus.Locked,
        "VAULT: cannot cancel"
    );

    uint256 shares = r.shares;
    if (s == WithdrawStatus.Pending) {
        _transfer(address(this), r.user, shares);
    } else {
        uint256 owed = r.owedUsdc;
        totalOwedUsdc -= owed;
        _mint(r.user, shares);
    }
    r.status = WithdrawStatus.Cancelled;
}

function setRouter(address newRouter) external override onlyOwner {
    require(newRouter != address(0), "VAULT: router=0");
    require(totalOwedUsdc == 0, "VAULT: owed not zero");
    require(pendingDepositsUsdc == 0, "VAULT: deposits pending");
    require(balanceOf(address(this)) == 0, "VAULT: withdrawals pending");
    emit RouterUpdated(router, newRouter);
    router = newRouter;
}

```

Impact

A shareholder can grief the vault by transferring a dust amount of shares directly to the vault. The orphaned balance has no corresponding withdrawal record, so it can never be locked, burned, or released through the router, and `setRouter()` reverts permanently with `VAULT: withdrawals pending`.

Proof of Concept

Initial state:

- Attacker owns at least 1 vault share unit.
- `totalOwedUsdc == 0`.
- `pendingDepositsUsdc == 0`.
- `balanceOf(address(vault)) == 0`.

Attack steps:

1. Attacker calls `vault.transfer(address[vault], 1)`.
2. The ERC20 transfer succeeds because `BuoyVault` does not reject transfers to itself.
3. `balanceOf(address[vault])` becomes `1`.
4. No `_withdrawals` entry is created and no router `globalId -> localId` mapping exists.
5. Owner calls `setRouter(newRouter)`.
6. The call reverts at `balanceOf(address[this]) == 0` with `VAULT: withdrawals pending`.

Recommendation

Do not use the vault's raw ERC20 share balance as the source of truth for pending withdrawals.

Track pending withdrawal shares explicitly with a `pendingWithdrawalShares` counter. Increment it in `acceptWithdrawalRequest`, decrement it in `lockOwed` and Pending `cancelWithdrawalRequest`, and gate `setRouter` on `pendingWithdrawalShares == 0`.

Team Response

Fixed.

M-04

MEDIUM

FIXED

Public Requests Can Indefinitely Invalidate Fulfiller NAV Snapshots

Description

Buoy settlements depend on an off-chain NAV snapshot submitted by the fulfiller together with the vault's current `interactionNonce`. This is intended to prevent stale NAV usage: if the vault state changes after the snapshot is computed, NAV-dependent settlement reverts.

The flaw is that public user actions, specifically deposit and withdrawal requests, also increment the same `interactionNonce` before any settlement occurs. Any user can front-run a pending fulfiller transaction with a minimum-sized `requestDeposit()` or valid `requestWithdrawal()`, causing the fulfiller's `expectedNonce` to become stale and forcing `fulfillDepositRequest()`, `lockOwed()`, or `publishNav()` to revert.

Location of Affected Code

File: [contracts/vault/Router.sol](#)

File: [contracts/vault/BuoyVault.sol](#)

```
// contracts/vault/Router.sol:358
function requestDeposit(
    address vault,
```

```

    address recipient,
    uint256 amount,
    uint256 minShares
) external payable override nonReentrant returns (uint256 globalId) {
    require(!_isVault(vault), "ROUTER: unknown vault");
    require(amount > 0, "ROUTER: amount=0");
    require(recipient != address(0), "ROUTER: recipient=0");
    require(msg.value == nativeDepositFee, "ROUTER: bad fee");
    ...
    uint256 localId = IBuoyVault(vault).acceptDepositRequest(...);
}

// contracts/vault/BuoyVault.sol:388
pendingDepositsUsdc += amount;
_bumpInteractionNonce();

// contracts/vault/Router.sol:514
function requestWithdrawal(
    address vault,
    uint256 shares,
    uint256 minOut
) external payable override nonReentrant returns (uint256 globalId) {
    ...
    uint256 localId = IBuoyVault(vault).acceptWithdrawalRequest(...);
}

// contracts/vault/BuoyVault.sol:526-527
_transfer(user, address(this), shares);
_bumpInteractionNonce();

// contracts/vault/BuoyVault.sol:439
require(expectedNonce == interactionNonce, "VAULT: stale snapshot");

// contracts/vault/BuoyVault.sol:573
_cacheNav(currentNav, expectedNonce);

// contracts/vault/BuoyVault.sol:1083-1087
function _cacheNav(uint256 currentNav, uint256 expectedNonce) internal {
    require(expectedNonce == interactionNonce, "VAULT: stale snapshot");
    latestNav = currentNav;
    latestNavTimestamp = uint64(block.timestamp);
    _bumpInteractionNonce();
}

// contracts/vault/Router.sol:466-467
require(
    vault.interactionNonce() == expectedNonce,
    "ROUTER: stale snapshot"
);

```

Impact

A motivated attacker can repeatedly delay core asynchronous settlement for a targeted vault:

- deposits remain pending and do not mint shares;
- withdrawals cannot progress from Pending to Locked;
- NAV heartbeats cannot update cached NAV;
- users may be forced to wait for timeout cancellation rather than normal settlement.

Proof of Concept

1. Vault `interactionNonce` is `N`.
2. The fulfiller computes NAV and submits one of:
 - `Router.fulfillDeposit(globalId, currentNav, N)` ;
 - `Router.fulfillDeposits(globalIds, currentNav, N)` ;
 - `Router.lockOwed(globalId, currentNav, N)` ;
 - `Router.publishNav(vault, currentNav, N)` .
3. An attacker observes the pending fulfiller transaction.
4. The attacker front-runs with `Router.requestDeposit(vault, attacker, minDepositUsdc, 0)` and pays the configured native fee.
5. `acceptDepositRequest` records the pending deposit and increments `interactionNonce` from `N` to `N+1`.
6. The fulfiller transaction lands with `expectedNonce = N` and reverts with `VAULT: stale snapshot` or `ROUTER: stale snapshot`.
7. The attacker repeats this for each observed fulfiller retry.
8. After the timeout, the attacker cancels pending deposits and recovers the USDC principal; recurring cost is gas plus the non-refundable request fee.

Recommendation

Do not use one nonce for both public request enqueueing and NAV-affecting settlement state.

Team Response

Fixed.

Unfulfillable User Requests Can Block Router Rotation

Description

Users fully control the slippage floors used when creating asynchronous vault requests: `minShares` for deposits and `minOut` for withdrawals.

For deposits, a malicious user can submit a valid request with an impossible `minShares` value. The vault accepts the request, pulls the USDC, stores it as Pending, and increments `pendingDepositsUsdc()`. Later fulfillment always reverts with `VAULT: slippage`, but the request remains Pending. After timeout, only the original payer can cancel the deposit.

This means a malicious payer can refuse to cancel an unfulfillable request and keep `pendingDepositsUsdc > 0` indefinitely. Since `setRouter()` requires `pendingDepositsUsdc == 0`, router rotation for that vault is blocked through the normal governance path.

The same design concern applies to withdrawals: a user-controlled impossible `minOut` can keep shares escrowed in the vault, and `setRouter()` also requires `balanceOf(address[this]) == 0`.

Location of Affected Code

File: [contracts/vault/Router.sol](#)

```
function requestDeposit(
    address vault,
    address recipient,
    uint256 amount,
    uint256 minShares
) external payable override nonReentrant returns (uint256 globalId) {
    ...
    uint256 localId = IBuoyVault(vault).acceptDepositRequest(
        msg.sender,
        recipient,
        amount,
        minShares
    );
}
```

File: [contracts/vault/BuoyVault.sol](#)

```
function acceptDepositRequest( address payer, address recipient, uint256 amount, uint256 minShares ) external
override onlyRouter nonReentrant whenNotPaused returns (uint256 localId) {
    // code
    IERC20Upgradeable[_asset].safeTransferFrom(
        msg.sender,
```

```

    address(this),
    amount
);
pendingDepositsUsdc += amount;
_bumpInteractionNonce();

_deposits[localId] = DepositRequest({
    payer: payer,
    recipient: recipient,
    amount: _toUint128(amount),
    minShares: _toUint128(minShares),
    createdAt: uint64(block.timestamp),
    status: DepositStatus.Pending
});
}

```

File: [contracts/vault/BuoyVault.sol](#)

```

function fulfillDepositRequest( uint256 localId, uint256 currentNav, uint256 expectedNonce ) external override
onlyRouter nonReentrant whenNotPaused returns (uint256 sharesToUser) {
    // code
    require(sharesToUser > 0, "VAULT: zero shares");
    require(sharesToUser >= minShares, "VAULT: slippage");

    r.status = DepositStatus.Fulfilled;
    pendingDepositsUsdc -= amount;
    _mint(recipient, sharesToUser);
    // code
}

```

File: [contracts/vault/BuoyVault.sol](#)

```

function cancelDepositRequest(
    uint256 localId,
    address caller,
    uint256 timeout
) external override onlyRouter nonReentrant returns (uint256 refundAmount) {
    DepositRequest storage r = _deposits[localId];
    require(r.status == DepositStatus.Pending, "VAULT: not pending");
    require(caller == r.payer, "VAULT: not payer");
    require(
        block.timestamp >= uint256(r.createdAt) + timeout,
        "VAULT: not timed out"
    );

    r.status = DepositStatus.Cancelled;
    refundAmount = r.amount;
    pendingDepositsUsdc -= refundAmount;
}

```

```
IERC20Upgradeable[_asset].safeTransfer(r.payer, refundAmount);
}
```

File: [contracts/vault/BuoyVault.sol](#)

```
function setRouter(address newRouter) external override onlyOwner {
    require(newRouter != address(0), "VAULT: router=0");
    require(totalOwedUsdc == 0, "VAULT: owed not zero");
    require(pendingDepositsUsdc == 0, "VAULT: deposits pending");
    require(balanceOf(address(this)) == 0, "VAULT: withdrawals pending");
    emit RouterUpdated(router, newRouter);
    router = newRouter;
}
```

Impact

A malicious user can indefinitely block router rotation for a vault by creating an unfulfillable pending request and refusing to cancel it after timeout.

Proof of Concept

Deposit-based attack:

1. Attacker approves USDC and calls `Router.requestDeposit(vault, attacker, amount, impossibleMinShares)`.
2. The router transfers USDC from the attacker and calls `acceptDepositRequest`.
3. The vault stores the request as Pending and increments `pendingDepositsUsdc`.
4. The fulfiller attempts to process the deposit.
5. `fulfillDepositRequest` computes `sharesToUser`, but `sharesToUser < minShares`.
6. The call reverts with `VAULT: slippage`.
7. The request remains Pending and `pendingDepositsUsdc` remains nonzero.
8. The request timeout passes.
9. Only the original payer can call `cancelDeposit`.
10. The attacker refuses to cancel.
11. The owner calls `setRouter(newRouter)`.
12. The call reverts with `VAULT: deposits pending`.

In a zero-supply vault, the attack is especially direct because the bootstrap branch mints `sharesToUser = amount`. Setting `minShares = amount + 1` is enough to make fulfillment impossible.

Withdrawal-based variant:

1. A shareholder calls `requestWithdrawal()` with an impossible `minOut`.
2. The vault escrows the user's shares at `address(this)`.
3. `lockOwed` reverts with `VAULT: slippage`.
4. The withdrawal remains Pending.
5. Only the original user can cancel after timeout.
6. If the user refuses, `balanceOf(address(this)) > 0`, so `setRouter()` reverts with `VAULT: withdrawals pending`.

Recommendation

Add a timeout cancellation path that does not depend on the malicious requester cooperating.

Team Response

Fixed.

M-06

MEDIUM

FIXED

Stale High-Water Mark Suppresses Fees After Vault Recapitalization

Description

When a vault is fully unwound, `totalSupply()` can return to zero while `highWaterMark` retains the value from the previous capitalization.

The next deposit then enters the zero-supply bootstrap branch and receives shares at a fresh 1:1 basis:

```
sharesToUser = amount;
```

However, `highWaterMark` is not reset to `PRICE_SCALE`.

If the previous capitalization had an HWM above `1e18`, the new capitalization can generate gains without paying performance fees until its new price per share exceeds that stale HWM.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function fulfillDepositRequest(uint256 localId, uint256 currentNav, uint256 expectedNonce) external override  
onlyRouter nonReentrant whenNotPaused returns (uint256 sharesToUser) {  
    // code  
    uint256 supply = totalSupply();
```

```

if (supply == 0) {
    require(totalOwedUsdc == 0, "VAULT: wound down");
    require(expectedNonce == interactionNonce, "VAULT: stale snapshot");

    sharesToUser = amount;

    latestNav = currentNav + amount;
    latestNavTimestamp = uint64(block.timestamp);
    _bumpInteractionNonce();
}
// code
}

```

The HWM is only updated during fee crystallization:

File: [contracts/vault/BuoyVault.sol](#)

```

function _crystallizeFee(uint256 currentNav, uint256 supply) internal {
    if (performanceFeeBps == 0) return;
    if (supply == 0 || currentNav == 0) return;

    uint256 net = _netNav(currentNav);
    if (net == 0) return;

    uint256 nps = (net * PRICE_SCALE) / supply;
    uint256 hwm = highWaterMark;

    if (nps <= hwm) return;

    // code

    uint256 newSupply = supply + feeShares;
    uint256 newHwm = (net * PRICE_SCALE) / newSupply;

    if (newHwm > hwm) {
        highWaterMark = newHwm;
    }
}

```

Neither a full unwind nor the zero-supply bootstrap resets `highWaterMark`.

Impact

After a full wind-down and recapitalization, the new vault cycle may under-collect performance fees until its price per share exceeds the HWM left by the previous capitalization.

Proof of Concept

Assume the previous capitalization has:

```
highWaterMark = 1.9e18
```

All shares and withdrawal obligations are later fully removed:

```
totalSupply = 0  
totalOwedUsdc = 0
```

The HWM remains:

```
highWaterMark = 1.9e18
```

A new depositor deposits 100 USDC.

The zero-supply branch gives:

```
100 USDC -> 100 shares  
PPS = 1.0
```

but the HWM is still:

```
1.9
```

If the new capitalization grows to:

```
NAV = 150 USDC  
supply = 100 shares  
PPS = 1.5
```

then `_crystallizeFee()` evaluates:

```
if (nps <= hwm) return;
```

Since:

```
1.5 <= 1.9
```

No performance fee is charged, even though the new capitalization gained 50%.

Recommendation

Reset the HWM when establishing a new zero-supply capitalization after all prior obligations are cleared:

```

if (supply == 0) {
  require(totalOwedUsdc == 0, "VAULT: wound down");
  require(expectedNonce == interactionNonce, "VAULT: stale snapshot");

  highWaterMark = PRICE_SCALE;

  sharesToUser = amount;

  latestNav = currentNav + amount;
  latestNavTimestamp = uint64(block.timestamp);
  _bumplInteractionNonce();
}

```

Team Response

Fixed.

M-07

MEDIUM

ACKNOWLEDGED

First-Depositor Share-Price Inflation via Missing Dead Shares at Bootstrap

Description

BuoyVault prices deposits as:

$$\text{sharesToUser} = (\text{amount} * \text{postSupply}) / \text{netNav};$$

where `netNav` represents the vault's NAV after accounting for `totalOwedUsdc`. Because the vault does not permanently lock any shares during bootstrap, the entire initial share supply can later be burned, allowing `totalSupply()` to be reduced to 1 wei.

An attacker can exploit this as follows:

1. Deposit 1,000 USDC as the first LP. The bootstrap branch mints 1,000,000 shares, establishing a 1 USDC-per-share price.
2. Withdraw all but 1 wei of the initial shares through the normal withdrawal flow. This reduces `totalSupply()` from 1,000,000 to exactly 1 wei.
3. Transfer 100k USDC directly to the vault contract. Because this transfer does not go through the protocol's deposit accounting, the donated USDC is counted as idle vault assets and increases `netNav`.

Once the supply has been reduced to 1 wei, the donation causes the share price to become arbitrarily large:

```
sharesToUser = (amount * 1) / netNav;
```

For any subsequent deposit smaller than the inflated `netNav` (now equal to 100k USDC), the calculation rounds down to zero.

The root cause is the absence of permanently locked "dead" shares at bootstrap. Without a non-burnable minimum supply, an initial depositor can collapse the share denominator to 1 wei and make the share price arbitrarily sensitive to direct NAV donations.

Additionally, anyone can `spotSend()` USDC directly to the vault clone's HyperCore address (equivalently, via CoreWriter Action 6 or Action 13 with `destination_dex = UINT32_MAX`).

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
// Bootstrap mint establishes the entire circulating supply.
// No permanently locked/dead shares are created.
if (postSupply == 0) {
    sharesToUser = 1_000_000;
    // code
}

// Deposit pricing becomes exploitable once totalSupply() is reduced to 1 wei.
sharesToUser = (amount * postSupply) / netNav;
require(sharesToUser > 0, "VAULT: zero shares");
```

Impact

New depositors are prevented from acquiring shares once the attacker has reduced the vault's supply to 1 wei and inflated `netNav` via a direct USDC donation. Any deposit below the resulting minimum threshold rounds down to zero shares and reverts, effectively blocking further deposits for ordinary users and preventing new capital from entering the vault.

The attacker does not need to permanently sacrifice the donated capital. The donation remains part of the vault's assets, backed by the attacker's remaining 1 wei of shares, which can subsequently be redeemed. The attack therefore requires only gas and a temporary capital lock-up.

Recommendation

Permanently lock a fixed amount of dead shares during the initial bootstrap mint so that `totalSupply()` can never fall below a meaningful minimum. These shares should be minted to an address incapable of redeeming or transferring them, such as `address(0)` or another permanently inaccessible sink.

As an additional defense-in-depth measure, the withdrawal path (`acceptWithdrawalRequest()` / `lockOwed()`) should reject any withdrawal that would reduce the circulating supply below a defined minimum threshold.

Team Response

Acknowledged.

L-01

● LOW

FIXED

Asynchronous Bridge Accounting Allows Deposits to Mint Excess Shares

Description

The vault prices fulfilled deposits from a fulfiller-supplied `currentNav`, while the production-style NAV calculation is:

`HyperCore balance + vault EVM USDC balance - pendingDepositsUsdc`

When a pending deposit is fulfilled, the vault removes the amount from `pendingDepositsUsdc`, mints shares, and sends the USDC through `CoreDepositWallet.deposit`. That bridge settles asynchronously. During the settlement gap, the funds have left the vault's EVM balance, have not yet appeared in HyperCore, and are no longer counted as pending.

A later deposit can therefore be fulfilled with a fresh nonce but an understated NAV, causing it to receive too many shares.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function fulfillDepositRequest(uint256 localId, uint256 currentNav, uint256 expectedNonce) external override
onlyRouter nonReentrant whenNotPaused returns (uint256 sharesToUser) {
    // code
    sharesToUser = (amount * postSupply) / netNav;
    _cacheNav(currentNav + amount, expectedNonce);

    r.status = DepositStatus.Fulfilled;
    pendingDepositsUsdc -= amount;
    _mint(recipient, sharesToUser);

    _bridgeAmount(amount);
    // code
}
```

File: [contracts/vault/BuoyVault.sol](#)

```
function _bridgeAmount(uint256 amount) internal {
    if (amount == 0) return;

    address depositWallet = coreDepositWallet;
    IERC20Upgradeable[_asset].safeApprove(depositWallet, 0);
    IERC20Upgradeable[_asset].safeApprove(depositWallet, amount);
    ICoreDepositWallet(depositWallet).deposit(
        amount,
        CORE_DEPOSIT_DEST_SPOT
    );
    emit BridgedToCore(amount);
}
```

The cited mainnet-style integration computes NAV as $hcE6 + evm - pending$, so it does not count fulfilled-but-unsettled outbound bridge amounts.

Impact

An attacker can time a deposit during the EVM-to-HyperCore bridge settlement gap and receive excess shares. This dilutes existing LPs and prior depositors.

Proof of Concept

Assume no fees and no owed withdrawals.

Initial state:

- HyperCore assets: 100 USDC
- Vault EVM USDC: 0
- `pendingDepositsUsdc`: 0
- Total supply: 100 shares

1. User A requests a 100 USDC deposit.

- Vault EVM USDC becomes 100.
- `pendingDepositsUsdc` becomes 100.
- Computed NAV remains $100 + 100 - 100 = 100$.

2. Fulfiller fulfills User A's deposit.

- User A receives $100 * 100 / 100 = 100$ shares.
- Total supply becomes 200.
- `pendingDepositsUsdc` is reduced to 0.
- `_bridgeAmount(100)` sends the USDC to `CoreDepositWallet`.

3. Before the bridge settles on HyperCore:

- HyperCore assets are still 100.
- Vault EVM USDC is 0.
- `pendingDepositsUsdc` is 0.
- Live computed NAV is understated at 100, despite 200 shares existing.

4. Attacker requests and fulfills a 100 USDC deposit during this gap.

- During the request, computed NAV is $100 + 100 - 100 = 100$.
- Fulfillment mints $100 * 200 / 100 = 200$ shares to the attacker.

5. After both bridges settle:

- Total assets are 300 USDC.
- Total supply is 400 shares.
- The attacker owns 200 shares, or 50% of the vault, after contributing only one third of the assets.

Nonce checks do not prevent this because the understated NAV can be freshly computed from current on-chain state; the missing state is the outbound bridge amount in transit.

Recommendation

Track outbound EVM-to-HyperCore bridge amounts as in-flight accounting.

Team Response

Fixed.

L-02

● LOW

FIXED

HIP-3 Transfer Path Hardcodes USDC for Non-USDC Collateral DEXes

Description

`sendAssetToHip3()` and `withdrawAssetFromHip3()` are exposed as generic HIP-3 DEX transfer helpers, but both always route through `_moveUsdcAcrossDex()`, which hardcodes `usdcTokenId` in the HyperCore Action 13 `sendAsset` payload.

This works only for HIP-3 DEXes whose collateral token is USDC. It cannot correctly fund or unwind HIP-3 DEXes collateralized by another token. The repository's own inspection scripts document the concrete example: `hyna` uses `USDE` token index `235`, while the vault sends token `0` /USDC.

Hyperliquid's `sendAsset` semantics require the transferred token to match the source/destination perp DEX collateral token. If the token does not match, the HyperCore-side transfer is rejected or no-ops, while the Buoy contract call can still emit a local success event.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function sendAssetToHip3(
    uint32 dexIndex,
    uint256 usdcAmount
) external override onlyRouter whenNotPaused {
    _moveUsdcAcrossDex(SPOT_DEX, dexIndex, usdcAmount);
    emit Hip3DexTransfer(dexIndex, usdcAmount);
}

function withdrawAssetFromHip3(
    uint32 dexIndex,
    uint256 usdcAmount
) external override onlyRouter whenNotPaused {
    _moveUsdcAcrossDex(dexIndex, SPOT_DEX, usdcAmount);
    emit Hip3DexWithdraw(dexIndex, usdcAmount);
}

function _moveUsdcAcrossDex(
    uint32 sourceDex,
    uint32 destinationDex,
    uint256 usdcAmount
) private {
    require(usdcAmount > 0, "VAULT: amount=0");
    uint256 hc = usdcAmount * (10 ** uint256(CORE_DECIMAL_OFFSET));
    require(hc <= type(uint64).max, "VAULT: transfer overflow");
    _sendAsset(
        address(this),
        address(0),
        sourceDex,
        destinationDex,
        usdcTokenId,
        uint64(hc)
    );
}
```

Impact

Authorized operators can call the HIP-3 transfer helpers for a non-USDC-collateral DEX and receive Buoy-side success events even though the HyperCore transfer cannot move the intended collateral.

Proof of Concept

Conceptual reproduction:

1. Vault has HyperCore spot USDC.
2. An authorized operator calls `Router.sendAssetToHip3(vault, hynaDexIndex, amount)`.
3. Router authorizes the caller and forwards to `BuoyVault.sendAssetToHip3()`.
4. The vault builds Action 13 with:
 - `sourceDex = SPOT_DEX`
 - `destinationDex = hynaDexIndex`
 - `token = usdcTokenId`
5. hyna requires collateral token `235` /USDE, not token `0` /USDC.
6. HyperCore rejects or no-ops the transfer because the Action 13 token does not match the DEX collateral.
7. The vault still emits `Hip3DexTransfer(dexIndex, usdcAmount)`, so the EVM-side transaction appears successful.

Recommendation

Do not treat all HIP-3 DEXes as USDC-collateralized.

Make these helpers explicitly USDC-only by validating `dexIndex` against a configured allowlist of USDC-collateral DEXes and reverting for non-USDC collateral, or generalize the helpers to accept the destination DEX's collateral token id instead of hardcoding `usdcTokenId`.

Team Response

Fixed.

L-03

LOW

FIXED

Zero NAV Cannot Be Published, Leaving Stale Positive Accounting After Total Loss

Description

`publishNav()` rejects `currentNav == 0`. Therefore, if a vault previously cached a positive NAV and later suffers a complete loss, the fulfiller cannot publish the correct zero NAV.

Because `totalAssets()`, `pricePerShare()`, and conversion views derive their values from `latestNav`, they continue reporting the previously cached positive NAV indefinitely.

There is no general alternative path for the fulfiller to update `latestNav` to zero after a trading loss.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol](#)

```
function publishNav(  
    uint256 currentNav,  
    uint256 expectedNonce  
) external override onlyRouter whenNotPaused {  
    require(currentNav > 0, "VAULT: nav=0");  
  
    uint256 supply = totalSupply();  
    if (supply > 0) {  
        _crystallizeFee(currentNav, supply);  
    }  
  
    _cacheNav(currentNav, expectedNonce);  
    emit NavPublished(currentNav, uint64(block.timestamp));  
}
```

The reporting views rely on the cached NAV:

File: [contracts/vault/BuoyVault.sol](#)

```
function totalAssets() external view override returns (uint256) {  
    return latestNav > totalOwedUsdc ? latestNav - totalOwedUsdc : 0;  
}
```

Similarly, `pricePerShare()`, `convertToAssets()`, and `convertToShares()` use `latestNav` as their accounting basis.

Impact

After a complete vault loss, the contract cannot represent its true zero NAV.

As a result:

- `totalAssets()` can remain artificially positive.
- `pricePerShare()` and conversion views can remain stale.
- The vault's public accounting can misrepresent an insolvent vault as still having value.
- NAV-sensitive operations requiring a fresh positive NAV cannot progress normally.

Proof of Concept

Assume the vault initially has:

```
totalSupply = 1,000 shares
latestNav = 1,000 USDC
totalOwed = 0
```

The vault then suffers a complete strategy loss:

```
actual gross NAV = 0
```

The fulfiller attempts to publish the correct NAV:

```
vault.publishNav(0, expectedNonce);
```

The call reverts:

```
VAULT: nav=0
```

Therefore:

```
latestNav remains 1,000 USDC
```

and:

```
vault.totalAssets(); // still returns 1,000 USDC
vault.pricePerShare(); // still reports positive value
```

despite the actual vault NAV being zero.

Recommendation

Allow `publishNav()` to cache a zero NAV:

```
function publishNav(
  uint256 currentNav,
  uint256 expectedNonce
) external override onlyRouter whenNotPaused {
  uint256 supply = totalSupply();

  if (supply > 0) {
    _crystallizeFee(currentNav, supply);
  }

  _cacheNav(currentNav, expectedNonce);
  emit NavPublished(currentNav, uint64(block.timestamp));
}
```

Team Response

Fixed.

L-04

LOW

ACKNOWLEDGED

Named HyperCore Agents Can Remain Authorized After Rotation

Description

`Router.registerCreAgent()` allows the fulfiller to register a HyperCore API wallet using an arbitrary non-empty `name`.

`BuoyVault.registerNamedAgent()` updates only the single `currentNamedAgent` address and then registers the new `(agent, name)` pair on HyperCore.

HyperCore treats different agent names as separate authorization slots. Therefore, registering a replacement agent under a different name does not revoke an agent previously registered under an older name.

As a result, `currentNamedAgent` can point to the latest agent while older named agents remain authorized on HyperCore.

Location of Affected Code

File: [contracts/vault/Router.sol](#)

```
function registerCreAgent(  
    address vault,  
    address agent,  
    string calldata name  
) external override onlyRole(FULLFILLER_ROLE) {  
    require(!_isVault(vault), "ROUTER: unknown vault");  
  
    IBuoyVault(vault).registerNamedAgent(agent, name);  
  
    emit CreAgentRegistered(vault, agent, name);  
}
```

File: [contracts/vault/BuoyVault.sol](#)

```
function registerNamedAgent(  
    address agent,  
    string calldata name  
) external override onlyRouter whenNotPaused {  
    require(agent != address(0), "VAULT: agent=0");  
    require(bytes(name).length > 0, "VAULT: name required");
```

```

address old = currentNamedAgent;
currentNamedAgent = agent;

_sendAddApiWallet(agent, name);

emit NamedAgentRegistered(old, agent, name);
}

```

The HyperCore registration includes the caller-supplied name:

File: [contracts/vault/BuoyVault.sol](#)

```

function _sendAddApiWallet(
    address apiWallet,
    string memory name
) internal {
    bytes memory encodedAction = abi.encode(apiWallet, name);
    bytes memory data =
        _buildAction(ACTION_ADD_API_WALLET, encodedAction);

    _sendRawAction(data);
}

```

Impact

A previously registered named agent may retain HyperCore trading authority after Buoy rotates `currentNamedAgent` to a new address under a different name.

If the private key of such a historical agent is later compromised, it may still be used to place or modify trades on behalf of the vault and potentially cause trading losses.

The issue is limited by the fact that agent registration requires the trusted `FULLFILLER_ROLE`, and old agents can be replaced by registering a controlled agent under the same historical name.

Proof of Concept

Assume the fulfiller initially registers:

```

router.registerCreAgent(vault, agentA, "agent-v1");

```

State becomes:

```

Buoy:
currentNamedAgent = agentA

HyperCore:
"agent-v1" -> agentA

```

The fulfiller later rotates to a new agent using a different name:

```
router.registerCreAgent(vault, agentB, "agent-v2");
```

Buoy now records:

```
currentNamedAgent = agentB
```

However, HyperCore authorization becomes:

```
"agent-v1" -> agentA  
"agent-v2" -> agentB
```

Because the new registration used a different name, `agentA` is not replaced and can remain authorized until its slot is explicitly overwritten, expires, or is otherwise removed by HyperCore.

Recommendation

Use a fixed name for the vault's CRE agent so every rotation replaces the same HyperCore authorization slot.

For example:

```
string internal constant CRE_AGENT_NAME = "buoy-cre-agent";  
  
function registerNamedAgent(  
  address agent  
) external onlyRouter whenNotPaused {  
  require(agent != address(0), "VAULT: agent=0");  
  
  address old = currentNamedAgent;  
  currentNamedAgent = agent;  
  
  _sendAddApiWallet(agent, CRE_AGENT_NAME);  
  
  emit NamedAgentRegistered(old, agent, CRE_AGENT_NAME);  
}
```

Team Response

Acknowledged.

L-05

LOW

FIXED

Enabling the Performance Fee Charges it Retroactively on All Prior Gains

Description

The `_crystallizeFee()` returns immediately when `performanceFeeBps == 0`, before the high-water mark is read or written. `highWaterMark` is set once in `initialize` to `PRICE_SCALE` (`1e18`, i.e. 1 USDC/share) and only ever advances inside this function.

So while the fee is disabled, the HWM is frozen at its initial value no matter how much the vault gains. The moment the owner sets a non-zero fee, the very next crystallization sees the entire lifetime gain as unrealized profit above the stale HWM and charges the fee on all of it.

Example: A vault launches with `performanceFeeBps = 0` and 1,000,000 shares at \$1.00/share (\$1M TVL). Over two years, the vault grows to **\$50.00/share** (\$50M TVL, a 4,900% gain), with `highWaterMark` frozen at `1e18` the entire time. The owner then calls `setPerformanceFeeBps(1000)` (10%) and the very next `fulfillDepositRequest()` or `lockOwed` triggers crystallization:

```
gainPerShare = 50e18 - 1e18 = 49e18 (over 1,000,000 shares)
fee owed = 10% × $49/share × 1,000,000 shares ≈ $4,900,000
```

The leader is minted fee shares worth roughly **\$4.9M** in a single transaction, extracted entirely from gains that accrued during the fee-free period. Existing LPs suffer approximately **9.8% immediate dilution** of their position value; an LP who held from day one and expected 0% fees is effectively charged a 10% retroactive tax on their entire profit.

This does not require malice to cause harm: an owner turning the fee on for the first time in good faith produces the same overcharge.

Location of Affected Code

File: [contracts/vault/BuoyVault.sol:1058-1067](#)

```
function _crystallizeFee(uint256 currentNav, uint256 supply) internal {
    if (performanceFeeBps == 0) return; // <-- HWM never advances
    if (supply == 0 || currentNav == 0) return;

    uint256 net = _netNav(currentNav);
    if (net == 0) return;
    uint256 nps = (net * PRICE_SCALE) / supply;
    uint256 hwm = highWaterMark;
    if (nps <= hwm) return;
    // code
}
```

Impact

LPs are diluted by fee shares on profits earned while the fee was contractually zero. Because `setPerformanceFeeBps()` is `onlyOwner` and capped at 50%, the owner enabling the fee after a sustained run-up causes the leader to capture a portion of the vault's entire historical gain in a single crystallization, with no on-chain mechanism to prevent or reverse it.

Recommendation

Keep the HWM tracking the peak even when no fee is charged. Advance the mark, then return:

```
function _crystallizeFee(uint256 currentNav, uint256 supply) internal {
    if (supply == 0 || currentNav == 0) return;
    uint256 net = _netNav(currentNav);
    if (net == 0) return;

    uint256 nps = (net * PRICE_SCALE) / supply;
    uint256 hwm = highWaterMark;
    if (nps <= hwm) return;

    if (performanceFeeBps == 0) {
        highWaterMark = nps; // no fee owed, but the peak still counts
        return;
    }
    // ... existing fee math unchanged
}
```

Team Response

Fixed.



Your security partner, for the long term

[SHIELDIFY.ORG](https://shieldify.org) · [X.COM/SHIELDIFYSEC](https://x.com/shieldifysec) · [LINKEDIN](#)